Assignment 2: Paper 2

On Large-Batch Training for Deep Learning:
Generalization Gap and Sharp Minima

Jireh Katebe

CS 6362 Advanced Machine Learning, Fall 2026
22 October 2026

1 Problem statement and understanding of the methodology

1.1 The problem and intuition

The authors investigate why training with larger batches can lead to poorer generalisation [1]. Larger batches allow more computation to happen in parallel, which could help speed up training. But small-batch and large-batch methods can reach similarly low training losses while performing differently on unseen data. The paper asks whether the shape of the loss around the solutions reached during training helps explain this difference.

Near a sharp solution, a small change in some directions through the model's weights can increase the loss quickly. Near a flat solution, the loss changes more gradually over the neighbourhood being examined. Small batches produce noisier gradient estimates, which the authors suggest help the optimiser escape sharp regions. Larger batches produce less noisy estimates and may settle in sharper regions. The experiments investigate whether these sharper solutions also have poorer test performance.

1.2 Mathematical formulation

Let $S = \{(x_i, y_i)\}_{i=1}^n$ contain n training examples drawn from a distribution $\mathcal{D}$, with labels $y_i \in \{1, \dots, K\}$. The network has parameters $w \in \mathbb{R}^P$ and predicts class probabilities $p_w(y \mid x)$. With cross-entropy $\ell_i(w) = -\log p_w(y_i \mid x_i)$, the empirical and expected losses are

$$L_S(w) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(w), \qquad R(w) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[-\log p_w(y \mid x)].$$

Training minimises L_S, while a held-out test set estimates performance on unseen examples. Following the minibatch formulation in the lectures [2], a batch B_t gives the gradient estimate

$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_w \ell_i(w_t).$$

The paper uses Adam to update w_t from these gradients, with the learning rate controlling the size of the updates. The averaging argument in Lecture 6 explains why independent gradient noise decreases as batch size increases. Batch normalisation (BN) and dropout complicate this idealised calculation because the gradients also depend on batch statistics and randomly dropped units.

Let w_{SB} and w_{LB} denote the small-batch and large-batch solutions. The problem is to understand why similar empirical losses can accompany different test performance, and whether local sensitivity of the loss helps account for that difference.

1.3 How the authors test it

The experiments use Adam for both methods, with 256 examples per small batch and 10% of the training set per large batch. Each setup is repeated from five random initialisations. The assigned Figure 3 panels are F1, a fully connected network on MNIST, and C1 and C2, shallow and deeper convolutional networks on CIFAR-10.

Figure 3 selects a pair of trained solutions and evaluates the path

$$w(\alpha) = (1-\alpha)w_{\mathrm{SB}} + \alpha w_{\mathrm{LB}}, \qquad \alpha \in [-1, 2].$$

At $\alpha = 0$ the weights equal the small-batch solution; at $\alpha = 1$ they equal the large-batch solution. Values outside $[0, 1]$ extrapolate beyond the endpoints. The models are already trained, so α specifies where to evaluate the loss along this path.

The horizontal axis shows α. The left vertical axis shows mean cross-entropy and the right shows classification accuracy. Solid curves use training data and dashed curves use test data. In the paper, the loss rises more steeply around the large-batch endpoint, while the small-batch endpoint lies in a broader low-loss region.

To examine directions beyond this path, the authors define the sharpness measure in their equation (4):

$$\phi_{w,L_S}(\epsilon, A) = 100\,\frac{\max_{z \in C_\epsilon} L_S(w + Az) - L_S(w)}{1 + L_S(w)},$$

$$C_\epsilon = \{z \in \mathbb{R}^r : |z_j| \le \epsilon(|(A^+ w)_j| + 1),\ j = 1, \ldots, r\}.$$

Here $A \in \mathbb{R}^{P \times r}$ selects the search directions and A^+ is its pseudoinverse. The authors approximate the maximum using $A = I_P$ for the full parameter space or random A with $r = 100$, at scales 10^{-3} and 5×10^{-4}. Large-batch solutions generally have higher scores and lower test accuracy. Starting large-batch training from small-batch checkpoints can reduce sharpness and improve test accuracy, suggesting that the training trajectory also matters.

2 Reproduction of Figure 3

The final figure uses a fresh seed-42 run of the full training notebook generated through ChatGPT, model Astra. The shared conversation records the identifier `gpt-6-astra-wm`.
Conversation: `https://chatgpt.com/share/6adab3d8-c2fc-83ea-8a39-08fb4b3fff4c`
GitHub repository: `https://github.com/Jireh1226/AdvancedML_assignements`
Paper folder: `Assignment_2/paper_2_large_batch/`
Run folder: `reruns/chat_seed42_v2_20261022/`
Executed notebook: `on_large_batch_training_chat_seed42_v2_bn_recalibrated.executed.ipynb`.

An earlier IDE-generated notebook had already produced six models before the chat. Those checkpoints were not reused in the fresh run. The new notebook trained six models, and the figure was subsequently recomputed from their saved weights after correcting the BN evaluation procedure. Execution, diagnostics and corrections used Codex/IDE assistance outside the shared chat. The authors' code had also been inspected for the earlier attempt before this rerun. These facts remain departures from the assignment's chat-only workflow and source-access sequence; the later run does not erase them.

2.1 Phase 1: understanding the paper and asking for code

The discussion checked the meaning of sharpness, the distinction between the optimisation step size and the path coordinate α, and which networks and datasets the assigned panels use. The request asked for F1, C1 and C2 using the supplied paper, with unspecified choices identified before implementation. Existing implementations were excluded from that request.

2.2 Phase 2: reviewing the LLM's output

The generated notebook included lengthy instructions and sharpness experiments for other tables. These were removed to retain only the assigned panels. Five seeds were reduced to the requested single seed, 42. FP64 evaluation and 121 path points were replaced with FP32 and 61 points as explicit compute choices, not values recovered from the paper.

The most consequential assumption concerned BN. The LLM recognised that affine extrapolation of variances can make them negative and proposed geometric interpolation instead:

$$v_\alpha = \exp((1-\alpha)\log v_{\mathrm{SB}} + \alpha \log v_{\mathrm{LB}})\,.$$

This keeps stored variances positive and preserves both complete endpoint models. It does not establish that those variances describe activations at intermediate or extrapolated weights. Full execution exposed extremely large losses away from the endpoints. A controlled evaluation later isolated this BN convention as a major cause.

The proposed fixed implementation hash was also insufficient: editing code without changing the string would leave the apparent identity unchanged. The executed version hashes the actual notebook code. The corrected analysis has a separate hash and checks that its training definitions and experiment plan match the archived notebook that produced the checkpoints.

Some choices were retained with qualifications. C1 uses asymmetric SAME padding, which the paper does not fully specify. Incomplete small batches are dropped, although the paper does not require that choice. The stopping rule checks loss every epoch and uses a fixed patience and cap; it is an operational rule, not proof of reaching a minimizer.

2.3 Phase 3: running, correcting, and comparing the results

2.3.1 Settings and validation

F1 uses all 60,000 MNIST training images and 10,000 test images, with five hidden layers of 512 units. C1 and C2 use all 50,000 CIFAR-10 training images and 10,000 test images. C1 has two 64-filter, 5×5, stride-two convolution stages, each followed by 3×3, stride-two max pooling, then dense hidden layers of 384 and 192 units. C2 has convolution groups of depths $(2, 2, 3, 3, 3)$ and widths $(64, 128, 256, 512, 512)$, pooling after each group, then a 512-unit hidden dense layer. All models have ten output logits.

Hidden affine layers are followed by BN and ReLU. C1's two dense hidden layers and C2's dense hidden layer use dropout 0.5; F1 has no dropout. Output logits have no BN. The parameter counts are 1,462,794 for F1, 283,274 for C1 and 14,991,946 for C2. C1 retains the GPT implementation's asymmetric SAME padding and a final $64 \times 2 \times 2$ feature map. The earlier notebook's symmetric padding gave the same dimensions but shifted receptive fields.

Each SB/LB pair starts from identical weights, verified by hashes. Initialisation is Xavier-uniform with zero biases; BN scale starts at one and shift at zero. Seed 42 provides repeatability, without implying that the authors used it. A single pair does not measure variation across initialisations.

Stated in the paper	Choice in the executed reproduction
Adam; SB 256; LB 10% of training data	Learning rate 0.001; coefficients $(0.9, 0.999)$; stabiliser 10^{-8}; no schedule or weight decay
Appendix B network sizes, BN and dense-layer dropout	Hidden BN before ReLU; SAME C1 padding; BN stabiliser 10^{-5} and momentum 0.1; paired seed 42
Training until loss ceases improving; five trials	One trial; evaluation each epoch; minimum 100, cap 300; patience 20 and improvement threshold 10^{-4}
Linear parameter path over $[-1, 2]$	61 points; FP32 evaluation; training-only BN recalibration at every point after an evaluation correction

Table 1: Paper specifications and implementation choices. The BN convention and stopping thresholds are not established by the paper.

Images are scaled to $[0, 1]$ without augmentation or further standardisation. Cross-entropy is computed from logits and averaged over images. Each large batch is one genuine forward/backward batch: 6,000 for F1 and 5,000 for C1/C2. Incomplete small batches are dropped, giving 234 updates per MNIST epoch and 195 per CIFAR-10 epoch, compared with ten per large-batch epoch. Gradient accumulation over small batches would change BN behaviour.

The gradient averaging and Adam updates follow Lecture 6, pages 38–42, and Lecture 6.1, pages 23–24 [2]. The lectures do not determine these numerical defaults or the stopping thresholds. Test accuracy does not control stopping. The saved final state is used rather than restoring the lowest-loss epoch.

Preflight checks covered architecture dimensions, paired initialisation, data transforms, batch contents and sampler states, exact short-run checkpoint continuation, and endpoint identities. Caching transformed tensors retained the supplied DataLoader's order and dropped remainder. One validation check initially failed because a comparison stopped one iterator before its final exhaustion step; exhausting both iterators fixed the test harness without changing training. A real 5,000-image C2 batch fitted ordinary FP32 execution, so no mixed precision or gradient accumulation was substituted.

The six fresh models and original geometric-BN paths completed in 65.2 minutes. Correcting the path evaluation took another 5.1 minutes and involved no optimiser updates.

2.3.2 Correcting the BN evaluation

All original path losses were finite, but this check alone missed the scale problem. At $\alpha = 2$, geometric extrapolation produced losses up to 8.79×10^{29}, compressing the other blue curves near zero on a linear axis. Those apparently flat curves were a display consequence of extreme measured values, not near-zero losses throughout the path.

An evaluation-only control tested $\alpha = 0, 0.5, 1, 2$ for each model pair. It first reproduced the saved values, then held trainable weights fixed and replaced only BN statistics. Each BN layer was reset and recalibrated on all training images in batches of 512, with dropout disabled. For a batch of m images following c images, the momentum was $m/(c + m)$. Statistics were then frozen for training and test evaluation; no test data entered calibration.

Model	Geometric BN	Training-only recalibration
	Training cross-entropy at $\alpha = 2$	
F1	2.69×10^{9}	4.096
C1	9.64×10^{5}	6.910
C2	8.79×10^{29}	9.294

Table 2: Same trainable weights, different BN statistics. No retraining was performed.

The difference also occurs between the endpoints. For C2 at $\alpha = 0.5$, training loss fell from 268.043 to 0.07314 and test accuracy rose from 23.77% to 82.82% after recalibration. This isolates BN handling as a major source of the changed curve. It does not show that the endpoint models had diverged.

The final figure uses this training-only convention, matching the earlier reproduction. All 183 path points were recomputed, with no smoothing, clipping or selection of favourable points. The twelve diagnostic points matched the recomputation; independent recalibrated endpoint checks passed. Trainable-parameter hashes were checked before and after each recalibration, and all six checkpoint file hashes remained unchanged.

Recalibration is a declared evaluation choice, not a known author setting. It changes endpoint BN buffers even when endpoint trainable weights are exact. Averaging batch variances is also not identical to computing a global activation variance, and deeper activations depend on calibration batches. Phase 4 describes the different convention in the released code.

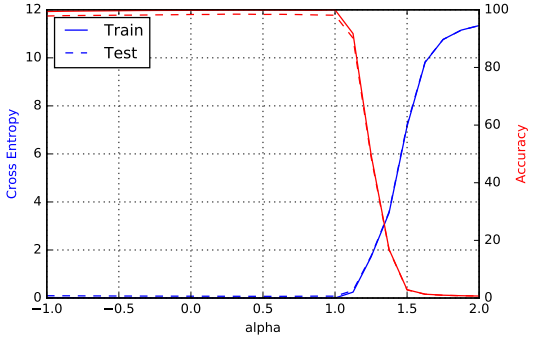
2.3.3 Results and differences from the paper

Network	Epochs SB/LB	Reproduced SB	Reproduced LB	Paper SB	Paper LB
		Test accuracy (%)			
F1	114/100	98.52	98.20	98.03 ± 0.07	97.81 ± 0.07
C1	159/111	71.24	66.68	80.04 ± 0.12	77.26 ± 0.42
C2	102/100	85.98	79.65	89.24 ± 0.12	87.26 ± 0.07

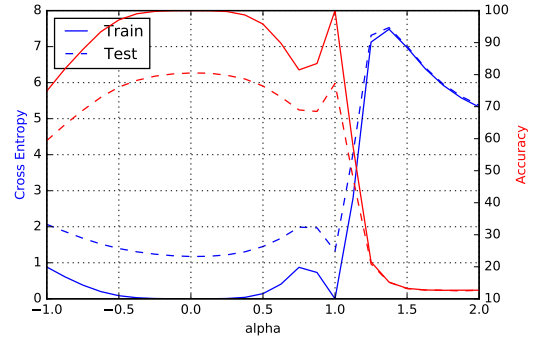
Table 3: Final recalibrated path endpoints and Table 2's five-trial means and standard deviations. This reproduction has one trial; the paper's means are not the particular pair shown in Figure 3.

Recalibrated training accuracies for SB and LB are 99.95% and 100% for F1, 99.89% and 99.26% for C1, and 99.90% and 99.79% for C2. SB has higher test accuracy in all three pairs, with gaps of 0.32, 4.56 and 6.33 percentage points. C1 is 8.80/10.58 points below the paper's means and C2 is 3.26/7.61 points below them. F1 is slightly above the reported means.

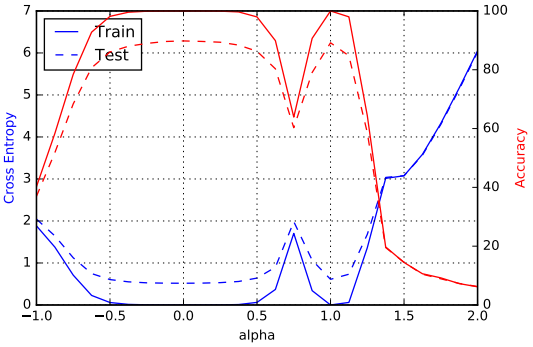
Original Figure 3



(a) F_1

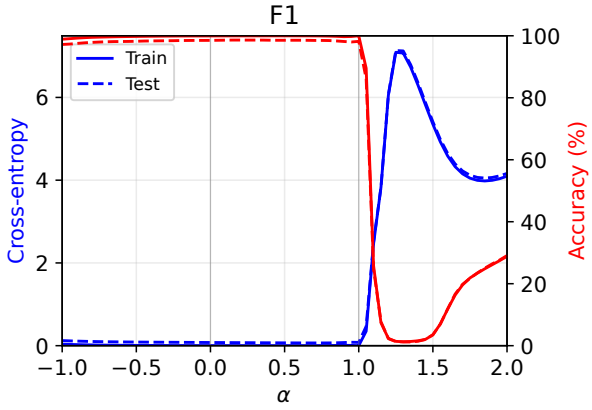


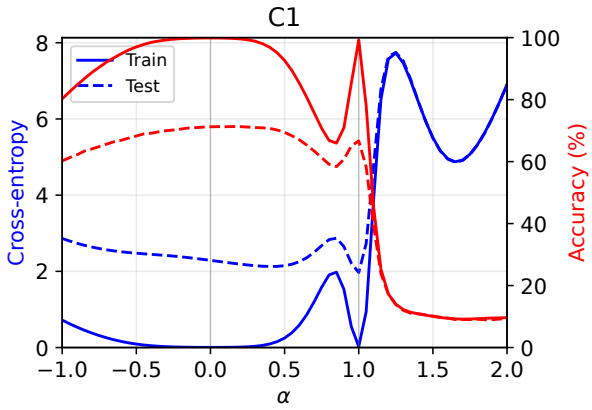
(c) C_1



(d) C_2

Final chat-based reproduction





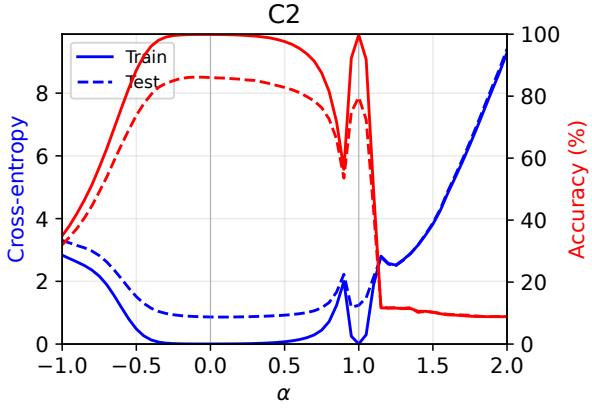


Figure 1: F1, C1 and C2, from top to bottom. Original panels are cropped from the supplied paper [1]; the reproduction uses the fresh models and corrected training-only BN evaluation. Blue is cross-entropy, red is accuracy, solid is training and dashed is testing. All 61 measured points are shown; loss scales differ between panels.

Figure 1 retains a broad low-training-loss region near SB and a steeper rise beyond LB, with an intervening barrier in C1/C2. This concerns one parameter direction under the stated BN convention, not all directions around either solution.

F1 still differs most clearly in its right-hand tail. In the paper, accuracy falls towards zero and stays low while cross-entropy rises towards approximately 11. Here test accuracy falls to 1.21% at $\alpha = 1.3$ and then recovers to 29.08% at $\alpha = 2$. Training cross-entropy peaks around 7.09 and ends at 4.10. The BN correction removes the extreme scale problem but does not remove this rebound.

C1 retains the barrier and sharp post-LB rise. Its training loss peaks around 7.73, closer in scale to the paper's roughly 7.5 peak than the earlier run, but falls and then rises again near the far-right end. Its test accuracy remains substantially lower. C2's test loss reaches 9.41 at $\alpha = 2$, compared with roughly 6 in the paper, and its accuracy drop occurs earlier. Paper-curve values here are visual approximations; reproduction values are saved measurements.

With the same recalibration procedure, five of the six new endpoint accuracies are within 0.24 points of the earlier run. C2 LB is 1.68 points lower. Training was not identical: the new C1 LB and C2 LB runs stopped after 111 and 100 epochs, compared with 300 and 180 earlier, and the batch ordering and C1 padding differ. These simultaneous changes prevent assigning the remaining difference to one cause.

The stopping labels need particular care. New C2 LB had its lowest recorded evaluation loss at epoch 59, but its final twenty epochs fluctuated before stopping at 100. New C1 LB stopped with stored-BN training accuracy 94.27%, which rose to 99.258% after recalibration without changing weights. Thus a patience rule can stop during variable BN-based evaluation, and a low stored-BN accuracy need not mean that the learned weights cannot fit the examples. The final states are retained rather than selected using test performance.

The raw stored-BN test accuracies were 98.53%/98.17% for F1, 70.51%/65.52% for C1 and 85.62%/77.18% for C2. These differ from the recalibrated values in Table 3. The experiment supports a qualitative SB/LB contrast but leaves the exact curve shapes and performance gaps incompletely reproduced.

2.4 Phase 4: comparison with the released implementation

The earlier IDE attempt was saved in local Phase 3 commit `8b28d5b` before author-code inspection. The fresh chat-based rerun happened after that inspection; it is not presented as a new source-blind Phase 3 attempt. The original `phase4_record.json` records the earlier notebook and inspected source versions, while the final rerun has its own manifests and correction record.

The comparison used the authors' repository at commit `8819876` [3]. Its Keras/Theano scripts cover C1–C4, with no F1 implementation. The separate PyTorch folder provides a preliminary VGG11 example. The Keras script agrees on scaling pixels by 1/255, Adam, batches of 256 and 5,000, and restoring the same initial weights before each batch regime. However, it trains for 20 epochs and evaluates 25 path points, while the paper describes loss-based stopping without a fixed budget.

Two architecture discrepancies remain. Released C1 omits a convolution stride argument, whose historical Keras default is one [4]; Appendix B.3 specifies stride two, followed by this reproduction. Released C2 adds convolutional dropout with probabilities 0.3 and 0.4 and dropout before its dense layer, beyond the dense-hidden dropout implemented here.

The released networks use Keras BN `mode=2`, computing input-batch statistics in both training and testing [4]. Their path evaluation uses batches of 5,000, including test batches. The submitted reproduction estimates statistics only on training images and freezes them for testing. Its geometric-versus-recalibrated diagnostic isolates a different contrast and does not measure the effect of the

authors' input-batch convention.

A controlled evaluation with the historical input-batch BN details could test that remaining difference without retraining. C1's stride and C2's extra dropout would require new paired training runs. Neither change was made for submission. No author code was executed or copied into the notebook, and no new seed or checkpoint was chosen to resemble the published figure.

2.5 Code and interaction record

The shared chat preserves the paper discussion and the full training proposal. The fresh run establishes that the proposal, with documented corrections, was actually trained; it is no longer only an unexecuted review of an earlier result. The IDE-assisted execution and BN correction still lie outside the shared conversation.

The run folder retains the supplied notebook text, original training input and executed notebook, six histories and manifests, original geometric-BN paths, diagnostic controls and the corrected executed notebook. Final report measurements are under `results_bn_recalibrated/`. The files `changes.md`, `inspection.md` and `bn_correction.md` distinguish training changes from evaluation-only changes. The report panels are regenerated from the final CSVs by `report_assets/render_panels.py`; their source hashes are recorded alongside them. The earlier notebook and measurements remain archived, with the final version identified by the repository README.

3 Critical review

3.1 Assessment of the paper

The paper's main strength is measuring loss geometry separately from test accuracy across several networks and datasets. This lets the authors compare the shape of the loss near the solutions with their performance on unseen data.

The starting point also matters. Each setup was repeated from five random initialisations, and the authors mention similar behaviour under other initialisation strategies, although they give little detail about those alternatives [1]. Table 2 shows lower mean large-batch test accuracy on all six networks; it does not establish that every individual run was worse. In Figure 5, starting large-batch training from later small-batch checkpoints improves test accuracy and reduces sharpness compared with earlier checkpoints. These results show that the gap depends on the training history. A systematic comparison of named initialisation schemes across paired seeds would help distinguish sensitivity to random starting weights from the benefit of preliminary small-batch training.

Several effects change alongside gradient noise when batch size changes. These include the number of updates per epoch and the statistics used by BN. Separating these effects would help test whether sharpness causes the performance gap. Further experiments could compare equal-example and equal-update budgets, control the normalisation convention, and test whether adding noise to large-batch updates changes sharpness and test accuracy together across several seeds.

The noise explanation also needs to account for Adam's memory of previous gradients. Lecture 6.1, pages 5–7 and 23–24, describes momentum and Adam's first- and second-moment estimates [2]. Momentum can sustain movement when the current gradient is small, but provides no general escape guarantee: with a zero gradient estimate and zero first-moment state, Adam takes no step. Both batch regimes in the paper already use Adam, and the authors report similar large-batch findings with other optimisers. A controlled comparison across momentum coefficients, with matched update budgets and learning-rate tuning, would help test how momentum affects the gap.

Lecture 4, pages 19–27, gives a way to connect sharpness to the rate at which the loss gradient changes [2]. Assuming a Lipschitz-continuous gradient within a neighbourhood gives

$$\|\nabla L_S(u) - \nabla L_S(v)\|_2 \leq L_\nabla \|u - v\|_2.$$

Where the loss is twice differentiable, L_∇ bounds the Hessian's operator norm. A direction of high positive curvature requires a larger bound, linking this condition to the Hessian discussion in Lecture 6.1, pages 9–11. For ReLU networks, this reasoning applies within smooth regions; global smoothness is not assured. The condition concerns sensitivity of the training loss, so a claim about generalisation still requires evidence on unseen data.

Figure 3 measures curvature along only one direction. For a fixed evaluation convention and $d = w_{\mathrm{LB}} - w_{\mathrm{SB}}$, the path's local second derivative is $d^\top \nabla^2 L_S(w) d$ wherever the loss is twice differentiable. Its apparent steepness depends on both the direction and its length. A flat section can coexist with sharp directions elsewhere, so this slice cannot determine a Lipschitz-gradient bound over the whole neighbourhood. The full-space and random-subspace searches partly address this limitation, although they remain approximate and depend on the permitted perturbation region.

Sharpness also depends on how the parameters are represented. In a ReLU network, multiplying a hidden unit's incoming weights and bias by a positive constant and dividing its outgoing weights by that constant can preserve the predictions while changing parameter-space distances and curvature. The paper's coordinate-scaled perturbation box addresses differences in weight magnitude, but does not establish invariance under all function-preserving changes. A comparison using controlled rescaling would help separate sensitivity of the predictor from sensitivity of its parameterisation.

The released C1/C2 code makes each prediction depend on the other images in its evaluation batch through BN statistics. The measured path therefore depends on evaluation batch size and composition as well as the weights. Comparing BN conventions would help separate evaluation effects from training effects. The stride and dropout discrepancies also leave uncertainty about which settings generated the published panels.

Exact padding, numerical optimiser settings and an operational stopping rule also need clarification. The five-trial summaries provide evidence beyond a single run, but Figure 3 displays one selected pair and does not show how much its shape varies across trials. Reporting all five paths with the exact run configuration would make the empirical claim easier to assess and reproduce.

Appendix B.4 also names C3 when introducing the deep ten-output architecture. The heading, Table 1 and surrounding text identify it as C2; C3 is the shallow CIFAR-100 model described in B.3. This appears to be a naming error rather than a different architecture, but it adds avoidable ambiguity for reproduction.

The authors also argue that their generalisation gap differs from the decline in test performance associated with continued training. Their displayed learning curves support that distinction. The reproduced C1 small-batch run did show such a decline, so excluding this explanation depends on the training setup.

3.2 Comparison with the OpenReview discussion

The public reviews also value the empirical contribution. Reviewer 1 emphasised its usefulness, while Reviewer 2 considered the evidence valuable despite limited novelty [5].

Reviewer 3 asked whether the endpoints were genuine local minima or only minima along one-dimensional slices. This overlaps with the concern above about interpreting a single path. The authors acknowledged that second-order optimality had not been checked, leaving uncertainty about the endpoints themselves as well as the directions examined around them.

Reviewer 2 challenged the claimed invariance to loss rescaling because the sharpness denominator contains $1 + L$. The authors withdrew that claim and reported similar results with an alternative denominator. That objection concerns changing the loss scale. The parameter-rescaling concern raised here concerns changes to the weights that preserve the network's predictions.

Reviewer 3 and public commenter Alex Lamb suggested noise injection. The authors reported no consistent test-error improvement from their attempted noise variants, leaving the role of noise unresolved. The update-budget and BN controls proposed here would help test the other effects of changing batch size.

References

[1] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *International Conference on Learning Representations*, 2017.

[2] Hirak Sarkar. *CS 6362 Advanced Machine Learning lecture slides*, Fall 2026. Supplied files `Lecture_4.pdf`, pp. 19–27; `Lecture_6.pdf`, pp. 38–42 and 50; and `Lecture_61.pdf`, pp. 5–7, 9–11 and 23–24.

[3] Nitish Shirish Keskar et al. *Large-batch training: released experiment code*. Repository at commit 8819876. Accessed 22 October 2026; file identities and recheck times are in `phase4_record.json`.

[4] Keras team. *Keras 1.2.2 historical layer implementations*. Batch normalisation and convolution layers. Used to interpret the released Keras 1.x API, not to establish the authors' exact installed version.

[5] OpenReview. Official reviews, author responses, and public discussion for paper `H1oyRlYgg`. Original discussion; accessed through archived OpenReview records, 22 October 2026. Reviews `S1Eq3CUEl`, `SJVFyZfNe`, `HkYQOlGEl`; questions and replies `ByV6OKkQg`, `HkcS-1U7l`, `HJWN7OkXe`, `SJdX9TrQg`, `H1UEbWm4x`, `SJMMtBgrg`, `H1oYDrgSe`.